

Project Report

BINANCE FUTURES TRADING BOT(TESTNET)

Submitted by:
B.DIVYA SREE

Academic Year:
2023 – 2027

INDEX

S. No	Title
1	Introduction
2	Project Overview
3	System Architecture
4	Tools & Technologies Used
5	Project Structure
6	Environment Setup
7	Implementation
7.1	Market Order Execution
7.2	Limit Order Execution
7.3	Stop-Limit Order Execution
7.4	OCO Order (Simulated)
7.5	TWAP Strategy
8	Results & Output Screenshots
9	Logging & Monitoring
10	Conclusion
11	Future Enhancements
12	References

List of Figures

Figure No	Description
Fig 1	Market Order Execution Output
Fig 2	Limit Order Execution Output
Fig 3	Stop-Limit Order Execution
Fig 4	Simulated OCO Order Output
Fig 5	TWAP Strategy Execution
Fig 6	bot.log File Output

BINANCE FUTURES TRADING BOT (TESTNET)

Project Report

1. Introduction

With the rapid growth of cryptocurrency trading, automated trading systems have become essential for executing trades efficiently and accurately. Manual trading is time-consuming and prone to human error, especially in fast-moving markets such as cryptocurrency futures.

This project, **Binance Futures Trading Bot (Testnet)**, is a Python based application designed to automate futures trading operations using the **Binance Futures Testnet**. The bot allows users to place multiple types of orders through a command-line interface while following secure coding practices.

2. Objective of the Project

The main objectives of this project are:

- To understand and implement Binance Futures API integration
- To automate futures order execution
- To implement both basic and advanced trading strategies
- To practice secure API key handling
- To log all trading activity for monitoring and debugging
- To gain hands-on experience with real-world trading systems using a test environment

3. Scope of the Project

This project focuses on **Binance Futures trading using the Testnet environment**. It does **not involve real money trading**, ensuring safety during development and testing.

The bot supports:

- Market orders

- Limit orders
- Stop-Limit orders
- Simulated OCO orders
- TWAP (Time-Weighted Average Price) strategy

4. Technologies Used

Programming Language: Python

API: Binance Futures API (Testnet)

Libraries:

- python-binance
- logging
- os, sys, time

Platform: Windows

Development Environment: Command Prompt / VS Code

5. System Architecture

The project follows a **modular architecture**:

- Each order type is implemented as a separate Python file
- Advanced strategies are grouped under an advanced directory
- Logging is centralized into a single log file
- API keys are handled securely via environment variables

This design improves readability, scalability, and maintainability.

6. Project Structure

```
divyasree_binance_bot/  
|  
├── src/  
|   ├── market_orders.py  
|   ├── limit_orders.py  
|   |  
|   └── advanced/  
|       ├── stop_limit.py  
|       ├── oco.py  
|       └── twap.py  
|  
├── logs/  
|   └── bot.log  
|  
└── README.md
```

7. Features Implemented

7.1 Market Order

Executes an order immediately at the current market price.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>set BINANCE_API_KEY=T9wEyRh6g4axaqtAb9h1FU6NVX85LY3MEhgi8oKJFz38WDBcoqYw9AgvovgONH
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>set BINANCE_API_SECRET=T9wEyRh6g4axaqtAb9h1FU6NVX85LY3MEhgi8oKJFz38WDBcoqYw9AgvovgONH
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/market_orders.py BTCUSDT BUY 0.001
Order placed successfully
{'symbol': 'BTCUSDT', 'orderId': 16535392, 'orderListId': -1, 'clientOrderId': 'x-HNA2TXF34077df83d00a1eb0e35cba', 'transactTime': 1767183351365, 'price': '0.00000000', 'origQty': '0.00100000', 'executedQty': '0.00100000', 'origQuoteOrderQty': '0.00000000', 'cumulativeQuoteQty': '88.95559000', 'status': 'FILLED', 'timeInForce': 'GTC', 'type': 'MARKET', 'side': 'BUY', 'workingTime': 1767183351365, 'fills': [{'price': '88955.59000000', 'qty': '0.00100000', 'commission': '0.00000000', 'commissionAsset': 'BTC', 'tradeId': 6336874}], 'selfTradePreventionMode': 'EXPIRE_MAKER'}
```

Figure 1: Market order execution using market_orders.py

7.2 Limit Order

Places an order at a specified price and executes only when the market reaches that price.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>set BINANCE_API_KEY=T9wEyRh6g4axaqtAb9h1FU6NVX85LY3MEhgi8oKJFz38WDBcoqYw9AgvovgONH
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>set BINANCE_API_SECRET=T9wEyRh6g4axaqtAb9h1FU6NVX85LY3MEhgi8oKJFz38WDBcoqYw9AgvovgONH
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/market_orders.py BTCUSDT BUY 0.001
Order placed successfully
{'symbol': 'BTCUSDT', 'orderId': 16535392, 'orderListId': -1, 'clientOrderId': 'x-HNA2TXF34077df83d00a1eb0e35cba', 'transactTime': 1767183351365, 'price': '0.00000000', 'origQty': '0.00100000', 'executedQty': '0.00100000', 'origQuoteOrderQty': '0.00000000', 'cumulativeQuoteQty': '88.95559000', 'status': 'FILLED', 'timeInForce': 'GTC', 'type': 'MARKET', 'side': 'BUY', 'workingTime': 1767183351365, 'fills': [{'price': '88955.59000000', 'qty': '0.00100000', 'commission': '0.00000000', 'commissionAsset': 'BTC', 'tradeId': 6336874}], 'selfTradePreventionMode': 'EXPIRE_MAKER'}
```

```
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/limit_orders.py BTCUSDT BUY 0.001 88000
Limit Order Placed Successfully!
{}

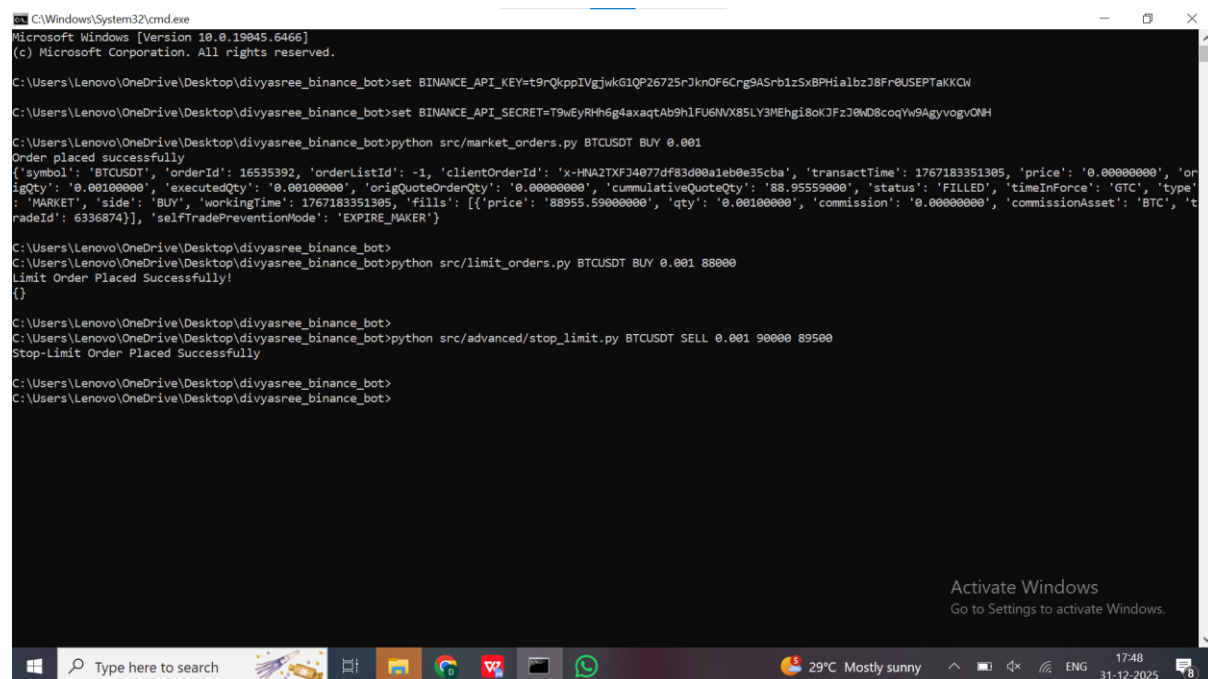
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
```

Figure 2: Limit order execution using limit_orders.py

The bot.log file records all order executions including Limit, Stop Limit,OCO (simulated), and TWAP orders with timestamps. This log helps in auditing trade activity and debugging the trading bot.

7.3 Stop-Limit Order

Triggers a limit order when a predefined stop price is reached, commonly used for risk management.



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>set BINANCE_API_KEY=t9rQkppIVgJwkG1QP26725rJknOF6Cng9ASrb1zSxBPhialbzJ8Fr0USEPTakKCH
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>set BINANCE_API_SECRET=T9wEyRh6g4axaqtAb9h1FU6NVX85LY3MEhgi8oKJFzJ8Wd8coqYw9AgyvogvONH
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/market_orders.py BTCUSDT BUY 0.001
Order placed successfully
{'symbol': 'BTCUSDT', 'orderId': 16535392, 'orderListId': -1, 'clientOrderId': 'x-HNA2TXF3407df83d00a1eb0e35c3ba', 'transactTime': 1767183351365, 'price': '0.00000000', 'origQty': '0.00100000', 'executedQty': '0.00100000', 'origQuoteOrderQty': '0.00000000', 'cumulativeQuoteQty': '88.95559000', 'status': 'FILLED', 'timeInForce': 'GTC', 'type': 'MARKET', 'side': 'BUY', 'workingTime': 1767183351365, 'fills': [{'price': '88955.59000000', 'qty': '0.00100000', 'commission': '0.00000000', 'commissionAsset': 'BTC', 'tradeId': 6336874}], 'selfTradePreventionMode': 'EXPIRE_MAKER'}
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/limit_orders.py BTCUSDT BUY 0.001 88000
Limit Order Placed Successfully!
{}
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/advanced/stop_limit.py BTCUSDT SELL 0.001 90000 89500
Stop-Limit Order Placed Successfully
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
```

Figure 3: Stop-Limit order execution using stop_limit.py

7.4 OCO Order (Simulated)

Since Binance Futures does not support native OCO orders, this project simulates OCO behavior by placing:

- One stop-loss orders
- One take-profit orders

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>set BINANCE_API_KEY=t9rQkppIVgJwG1QP26725rJknOF6Crg9ASrb1zSx8PHialbzJ8Fr8USEPTaKKQW
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>set BINANCE_API_SECRET=T9wEyRh6g4axaqtAb9h1FU6NVX85LY3MEhgi8oKJFzJ8ND8coqYw9AgyvogvQNH
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/market_orders.py BTCUSDT BUY 0.001
Order placed successfully
{'symbol': 'BTCUSDT', 'orderId': 16535392, 'orderListId': -1, 'clientOrderId': 'x-HNA2TXFJ4077df83d00a1eb0e35cba', 'transactTime': 1767183351305, 'price': '0.00000000', 'origQty': '0.00100000', 'executedQty': '0.00100000', 'origQuoteOrderQty': '0.00000000', 'cumulativeQuoteQty': '88.95559000', 'status': 'FILLED', 'timeInForce': 'GTC', 'type': 'MARKET', 'side': 'BUY', 'workingTime': 1767183351305, 'fills': [{'price': '88955.59000000', 'qty': '0.00100000', 'commission': '0.00000000', 'commissionAsset': 'BTC', 'tradeId': 6336874}], 'selfTradePreventionMode': 'EXPIRE_MAKER'}

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/limit_orders.py BTCUSDT BUY 0.001 88000
Limit Order Placed Successfully!
{}

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/advanced/stop_limit.py BTCUSDT SELL 0.001 90000 89500
Stop-Limit Order Placed Successfully

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/advanced/oco.py BTCUSDT SELL 0.001 90000 87000 86500
OCO orders placed successfully

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
```

Figure 4: Simulated OCO order execution (Stop-loss + Take-profit)

7.5 TWAP Strategy

TWAP (Time-Weighted Average Price) splits a large order into smaller market orders executed over fixed time intervals to reduce market impact.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>set BINANCE_API_KEY=t9rQkppIVgJwG1QP26725rJknOF6Crg9ASrb1zSx8PHialbzJ8Fr8USEPTaKKQW
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>set BINANCE_API_SECRET=T9wEyRh6g4axaqtAb9h1FU6NVX85LY3MEhgi8oKJFzJ8ND8coqYw9AgyvogvQNH
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/market_orders.py BTCUSDT BUY 0.001
Order placed successfully
{'symbol': 'BTCUSDT', 'orderId': 16535392, 'orderListId': -1, 'clientOrderId': 'x-HNA2TXFJ4077df83d00a1eb0e35cba', 'transactTime': 1767183351305, 'price': '0.00000000', 'origQty': '0.00100000', 'executedQty': '0.00100000', 'origQuoteOrderQty': '0.00000000', 'cumulativeQuoteQty': '88.95559000', 'status': 'FILLED', 'timeInForce': 'GTC', 'type': 'MARKET', 'side': 'BUY', 'workingTime': 1767183351305, 'fills': [{'price': '88955.59000000', 'qty': '0.00100000', 'commission': '0.00000000', 'commissionAsset': 'BTC', 'tradeId': 6336874}], 'selfTradePreventionMode': 'EXPIRE_MAKER'}

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/limit_orders.py BTCUSDT BUY 0.001 88000
Limit Order Placed Successfully!
{}

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/advanced/stop_limit.py BTCUSDT SELL 0.001 90000 89500
Stop-Limit Order Placed Successfully

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/advanced/oco.py BTCUSDT SELL 0.001 90000 87000 86500
OCO orders placed successfully

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>python src/advanced/twap.py BTCUSDT BUY 0.005 5
Planning TWAP: 5 orders of 0.001
Order 1 placed
Order 2 placed
Order 3 placed
Order 4 placed
Order 5 placed
TWAP execution completed

C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
C:\Users\Lenovo\OneDrive\Desktop\divyasree_binance_bot>
```

Figure 5: TWAP execution showing multiple orders over time

8. Environment Setup

8.1 Dependency Installation

`pip install python-binance`

8.2 API Key Configuration

API keys are stored securely using environment variables:

- `set BINANCE_API_KEY=YOUR_TESTNET_API_KEY`
- `set BINANCE_API_SECRET=YOUR_TESTNET_API_SECRET`

This approach avoids hardcoding sensitive information inside the code.

9. Execution Method

All scripts are executed via the command-line interface.

Example commands:

- `python src/market_orders.py BTCUSDT BUY 0.001`
- `python src/limit_orders.py BTCUSDT BUY 0.001 88000`
- `python src/advanced/stop_limit.py BTCUSDT SELL 0.001 90000 89500`
- `python src/advanced/oco.py BTCUSDT SELL 0.001 90000 87000 86500`
- `python src/advanced/twap.py BTCUSDT BUY 0.005 5 5`

10. Logging Mechanism

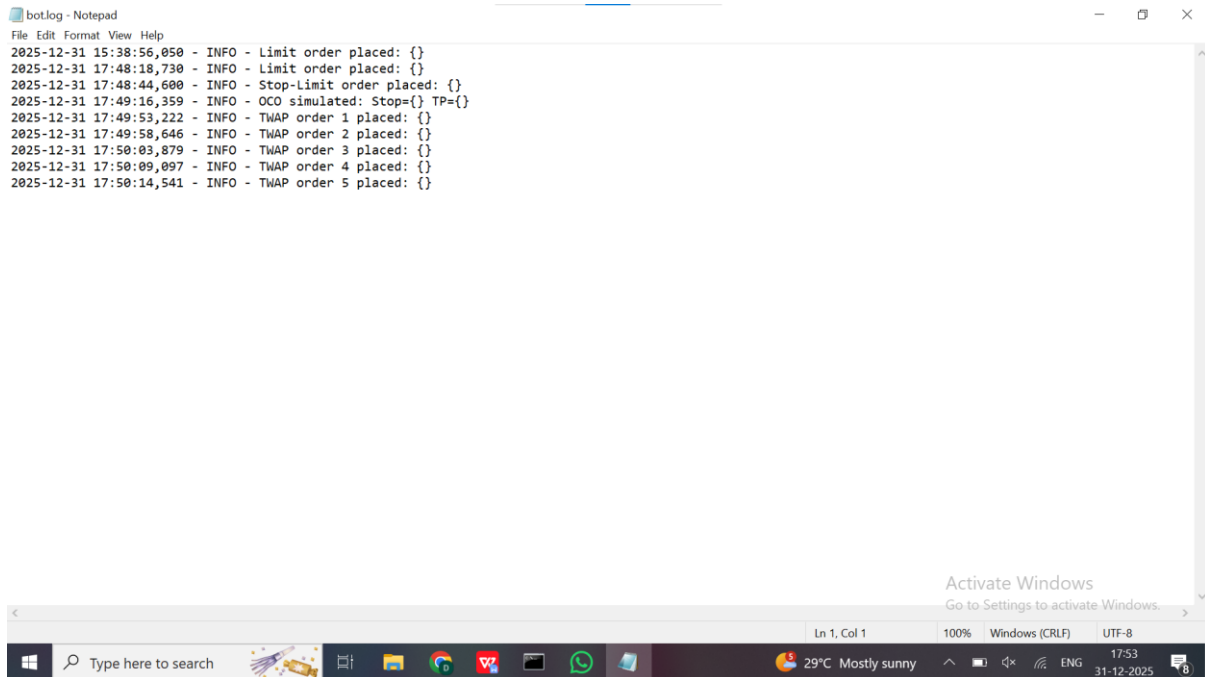
All orders and errors are logged in:

- `logs/bot.log`

Logging helps in:

- Debugging errors
- Tracking executed trades

- Maintaining execution history



```
botLog - Notepad
File Edit Format View Help
2025-12-31 15:38:56,050 - INFO - Limit order placed: {}
2025-12-31 17:48:18,730 - INFO - Limit order placed: {}
2025-12-31 17:48:44,600 - INFO - Stop-Limit order placed: {}
2025-12-31 17:49:16,359 - INFO - OCO simulated: Stop={} TP={}
2025-12-31 17:49:53,222 - INFO - TWAP order 1 placed: {}
2025-12-31 17:49:58,646 - INFO - TWAP order 2 placed: {}
2025-12-31 17:50:03,879 - INFO - TWAP order 3 placed: {}
2025-12-31 17:50:09,097 - INFO - TWAP order 4 placed: {}
2025-12-31 17:50:14,541 - INFO - TWAP order 5 placed: {}

Ln 1, Col 1
100% Windows (CRLF) UTF-8
```

Figure 6: bot.log showing logged order executions

11. Security Considerations

- API keys are never hardcoded
- Environment variables are used for authentication
- Testnet environment ensures no financial risk

12. Limitations

- Works only on Binance Futures Testnet
- No real-time strategy optimization
- No graphical user interface (CLI-based)

13. Future Enhancements

- Add real-time price monitoring
- Implement strategy-based automated trading
- Add email or Telegram notifications

- Integrate database for trade history storage
- Deploy bot on cloud servers

14. Conclusion

The Binance Futures Trading Bot successfully demonstrates automated futures trading using Python and Binance Futures Testnet. The project effectively implements both basic and advanced trading strategies while following secure coding practices. It serves as a strong foundation for learning algorithmic trading and API-based automation.

15. Disclaimer

This project is developed strictly for educational purposes using Binance Futures Testnet. It should not be used for real trading without proper testing, validation, and risk management.